

Reviewer Pack — Smallest Singular Value Inverse Proportional to SOS Degree f...

Entry b35e8c0854e4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 16:48:41 UTC

Smallest Singular Value Inverse Proportional to SOS Degree for Max-CUT

Entry ID: b35e8c0854e4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 16:48:41 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random graph $G(n, p)$ with adjacency matrix A , the smallest singular value $\sigma_{\min}(A)$ satisfies $\sigma_{\min}(A) \geq c * (d)^{-1}$, where d is the SOS degree required to approximate Max-CUT with ratio $0.878 \pm \varepsilon$ for $n \leq 40$.

Rationale (proposer's reasoning):

Spectral properties of random matrices influence semidefinite programming relaxations. The inverse relationship between $\sigma_{\min}$ and SOS degree suggests that graphs with poor spectral concentration require higher-degree SOS proofs to achieve optimal approximations.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d9b2cf5a4fdf4fe6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 40
    p = 0.5
    c = 0.1

    random.seed(seed)

    def generate_random_graph(n, p):
        A = [[random.random() < p for _ in range(n)] for _ in range(n)]
        for i in range(n):
            A[i][i] = 0
        return A

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i + 1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return [abs(A[i][n-1]) for i in range(m)]

    def compute_sos_degree(d):
        # Placeholder function to simulate SOS degree
        return d

    def compute_smallest_singular_value(A):
```

```

    U, S, Vt = gaussian_elimination(A)
    return min(S)

A = generate_random_graph(n, p)
sigma_min = compute_smallest_singular_value(A)

for d in range(1, 11):
    sos_degree = compute_sos_degree(d)
    if sigma_min < c * (d ** -1):
        return {
            "metric_name": "sigma_min",
            "metric_value": sigma_min,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"SOS degree {d} does not satisfy the conjecture"
        }

    return {
        "metric_name": "sigma_min",
        "metric_value": sigma_min,
        "instances_tested": 10,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"SOS degree {first_failing_seed} does not satisfy the conjecture\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ff70dfc.py", line 88, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ff70dfc.py", line 58, in run_trial
    sigma_min = compute_smallest_singular_value(A)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ff70dfc.py", line 54, in compute_small
    U, S, Vt = gaussian_elimination(A)
    ^^^^^^^
```

ValueError: too many values to unpack (expected 3)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during execution, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the Gaussian elimination implementation to return exactly 3 values and rerun tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	54780
2	preregistration	ollama_remote	qwen3:8b	0	0	25057
3	novelty	ollama_remote	qwen3:8b	0	0	21150
4	novelty	ollama_remote	qwen3:8b	0	0	13600
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12844
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9727
7	judge	ollama_remote	qwen3:8b	0	0	27080

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 164238 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/b35e8c0854e4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/b35e8c0854e4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b35e8c0854e4.tar.gz` (if generated)